PetApp mini-game architecture

How mini-games are made, published, delivered and run on iOS. October 2026.

Deliverables

Item	Where, or the short answer
Git repository	github.com/aliameenrana/petapp
Screen recording	Full flow on iOS (Google Drive)
One-page README	README.md in the repository root. The answers below summarise it.
Project structure and key packages	Four folders: the app (`PetAPP`), a creator project (`MiniGameStudio`), the shared `MiniGameSDK` package, and the live service (`LiveServer`). Puerts runs game logic as interpreted TypeScript, AssetBundles carry game art, UI Toolkit and DOTween drive the UI, and a Cloudflare Worker with R2 serves games. Sections 1, 2 and 5.
Mini-game approach	Each game is a zip per platform with `game.js` and an AssetBundle, stored under its hash. The app downloads it on first open, verifies it, caches it on device, and offers updates when the catalog shows a newer version. Alternatives were C# hot-patching, a WebView, Jint and Luau. iOS forbids JIT, downloaded native code, and exposing native APIs to downloaded code. Sections 2, 3 and 6.
Adding a game, and UGC	Add a folder with `game.json`, `game.ts` and its assets to a creator project, then **Publish to live**. Players get it within seconds, with no app release. Creators of UGC install the same SDK package; the sandbox already treats game code as untrusted. Sections 4 and 7.
Feed loading and preloading	The feed list comes from the catalog. Three pooled players cover the visible clip and its neighbours, each downloaded to a local cache before playing, so the next clip is ready before the swipe. Pages show progress, a timeout after 15 s without progress, and Retry on failure. Clips fill or letterbox depending on their shape.
Video call	Live change: edit `game.ts` in MiniGameStudio, publish, and the running app offers the update within 5 seconds.

1. The pieces

A mini-game is not part of the app. It is built in a separate Unity project, published to a server, and downloaded the first time a player opens it. Updates arrive the same way, with no App Store release.

- **MiniGameSDK**, a Unity package. It is the contract between a game and the app, and both sides install it.
- **A creator's Unity project**, where a game's code, prefabs, effects and sounds live, and where it is published from.
- **The live service**, a Cloudflare Worker with R2 storage, deployed at a public web address (currently aliameen.com/petapp).
- **PetApp**, the iOS app. It lists games, downloads and caches them, and runs them in a sandbox.

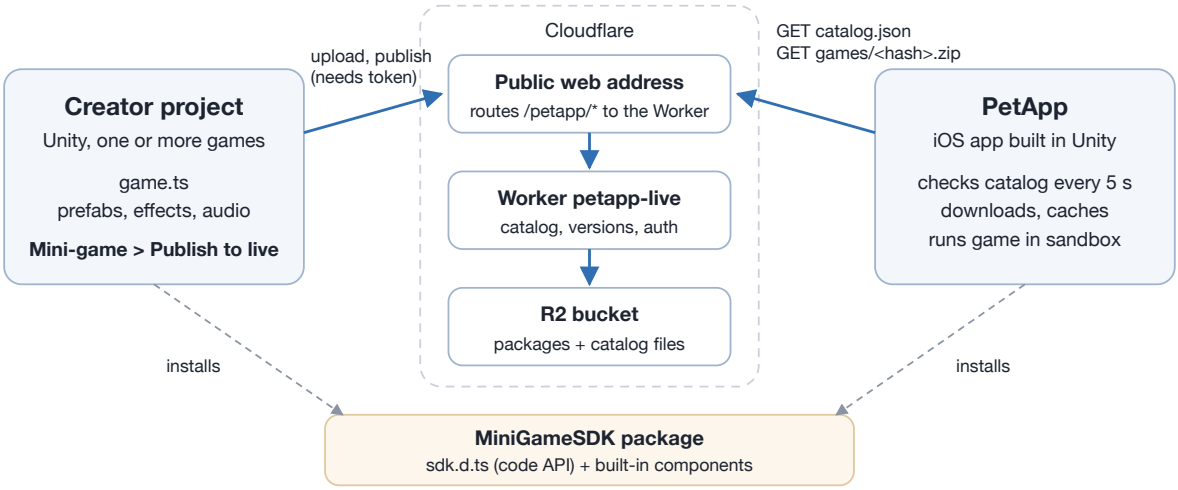


Figure 1. Creator projects and the app never talk directly. They meet at the live service and share the SDK.

A published game is a zip of two files per platform: `game.js` (the logic, compiled from TypeScript) and `assets.bundle` (a Unity AssetBundle of prefabs, materials, textures and sounds). The iOS zip for Catch the Prize is about 3.4 MB.

2. Why TypeScript on Puerts, not Lua or Luau

Downloaded logic can't be C#. Unity compiles C# for iOS ahead of time with IL2CPP, so a shipped app can't load new C#. The logic has to run in an interpreter already inside the app, with no JIT, in a sandbox we control.

	TypeScript on Puerts (QuickJS)	Luau	Lua 5.x (xLua, MoonSharp)
Types	Full static types, checked before publishing, from one `.d.ts` file.	Gradual types.	None.
Tooling	VS Code, `tsc` and npm work with no setup.	Roblox Studio, or a separate language server.	Basic editor support.
AI assistance	One of the most common languages in public code. Coding agents write it well.	A Roblox dialect with far less public code to learn from.	Less common than TypeScript.
Sandbox	Closed by the SDK (section 4).	Built in.	By hand.

We chose TypeScript for what it gives creators every day. Type safety comes first. The whole SDK is one `.d.ts` file, so every call is checked and autocompleted, and the publish tool generates a type listing every asset in the game. A typo like `sdk.spawn("bmob")` fails at compile time in the editor, not on a player's phone. Most developers already know JavaScript or TypeScript, and the standard tools work out of the box.

TypeScript also suits how games are now written. It is one of the most common languages in public code, which is what LLMs and coding agents learn from, so they produce correct TypeScript far more reliably than Luau. For UGC, where many creators will build with AI help, that directly affects how fast games get made and how often they work the first time.

Puerts makes this practical in Unity. It is maintained, ships prebuilt for iOS, Android and the editor, and runs on QuickJS, a pure interpreter that iOS allows. Luau has no maintained Unity binding, so it would mean building and maintaining a native plugin for every platform before writing a single game. Raw speed is not a concern here: scripts only make decisions, while rendering, physics, particles and audio run in Unity. And because the host only ever sees JSON commands, the app is not tied to any one language.

3. What iOS allows

App Store Review Guideline 2.5.2 sets the general rule:

> Apps should be self-contained in their bundles, and may not read or write data outside the designated container area, nor may they download, install, or execute code which introduces or changes features or functionality of the app, including other apps.

Guideline 4.7 makes an exception: "Apps may offer certain software that is not embedded in the binary, specifically HTML5 and JavaScript mini apps and mini games". Its condition 4.7.2 shapes our sandbox: "Your app may not extend or expose native platform APIs or technologies to the software without prior permission from Apple." The Developer Program License Agreement allows downloaded interpreted code as long as it doesn't change the app's main purpose, bypass OS security, or turn the app into a store for other apps.

Allowed	Not allowed
• JavaScript run by an interpreter shipped in the app (`game.js` in QuickJS). • Data and art (`assets.bundle`). • Mini games that fit the app's purpose.	• Native code, and new C# at runtime. This is why prefabs can only use components the app already contains. • JIT compilation. • Exposing native APIs (camera, contacts, files) to downloaded code.

Before launching player-made games, 4.7 also requires content reporting and user blocking (4.7.1), an index of games with universal links (4.7.4), and age-gating (4.7.5). The prototype doesn't have these yet.

4. The SDK package

The SDK is a Unity package (`com.petapp.minigame-sdk`). A creator imports it into any Unity 6000.0.68f1 project using URP 17.0.4, through Package Manager or as a `.unitypackage`. The engine version and render settings must match the app, because AssetBundles only load in the version they were built with.

Today the package is set up in one project, MiniGameStudio, which holds one game. Creators making UGC would keep a project per game, or one project for several games that share scripts. The publish tool (**Mini-game > Publish to live**) is an editor script in that project today and would ship inside the package for creators.

```
Assets/Games/catch-the-prize/
├── game.json        id, title, sdk version (the server assigns the game version)
├── game.ts          the game's logic
├── Assets/          names the game can load: coin.prefab → sdk.spawn("coin", ...)
└── Source/          materials, textures, models; packaged only if a prefab uses them
```

Prefabs may use Unity's built-in components and SDK components. A custom C# script on a prefab won't run in the app, because the app doesn't contain it.

What a game can call

Function	What it does
`onStart()`	Called once by the host. After that the game reacts to timers and events.
`sdk.spawn(key, {x, y})`, `sdk.pet`	Create a prefab, or get the player's pet. Both return a handle.
`handle.x`, `.y`, `.velocity`, `.play(clip)`, `.destroy()`	Move, animate (jump, happy, sad) or remove an object.
`handle.onTouch(other, fn)`, `.onOffscreen(fn)`	React to overlaps and to objects leaving the screen.
`sdk.timer.after`, `every`, `cancel`	Timers.
`sdk.input.onTap`, `onDrag`	Touch input in world coordinates.

`sdk.ui.setScore`, `showTimer`, `banner`	The HUD.
`sdk.audio.play`, `music`, `sdk.camera.use`, `shake`	Sound, music, camera preset and shake.
`sdk.time`, `sdk.random()`, `sdk.log()`	Clock, host-seeded random numbers, console.
`sdk.endGame({score})`	Ends the session and reports the score. The app decides the coin payout.

The package also holds built-in components that prefabs can use. Today there is one, `HostMotion`, which spins, bobs or drifts an object. A prefab stores only a reference to it and the app supplies the code, so a new component needs an app release before games can use it.

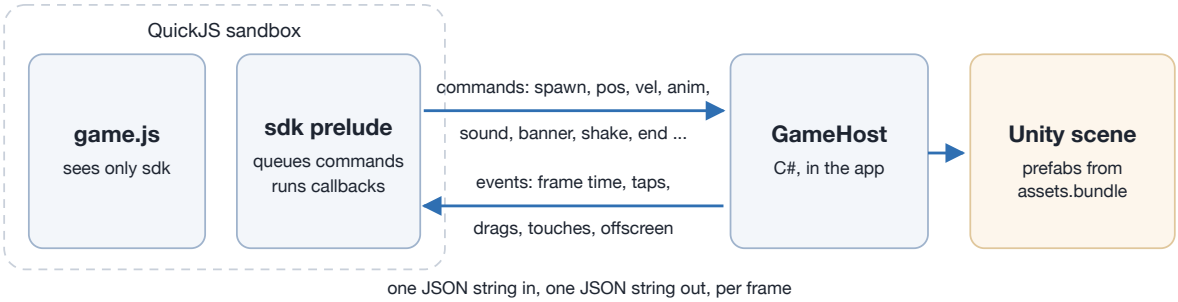


Figure 2. The script never holds a C# object. Everything it does reaches the host as a plain command.

Before a game runs, the prelude deletes every non-standard global, including Puerts' bridges to C# (`CS`, `puer`, `require`), and freezes `sdk`. If a global can't be removed, the game doesn't start. An automated test checks this.

5. The live service

A Cloudflare Worker (`petapp-live`) with an R2 bucket, deployed at a public web address (currently aliameen.com/petapp). The Worker has no address of its own; the site forwards `/petapp/*` to it through a service binding. It is deployed with `npx wrangler deploy` and runs on Cloudflare, not on anyone's laptop.

Route	What it does
`GET /catalog.json`	Every game's current version and package hashes, plus the video feed. Never cached.
`GET /games/<sha256>.zip`	A package. The URL is the file's hash, so it can be cached forever.
`PUT /api/packages/<sha256>`	Upload a zip. R2 rejects it if the bytes don't match the hash.
`POST /api/publish`	Make uploaded packages the game's next version.

- `/api/` routes need a bearer token, stored as a Worker secret. The publish tool asks for it once per editor session and keeps it in memory only.
- The server assigns version numbers. It only replaces the catalog if nothing changed since it read it, so two publishes can't claim the same version.
- The app checks every download against the catalog's hash, and refuses zip entries that would land outside the game's folder.

6. From a change to the player's screen

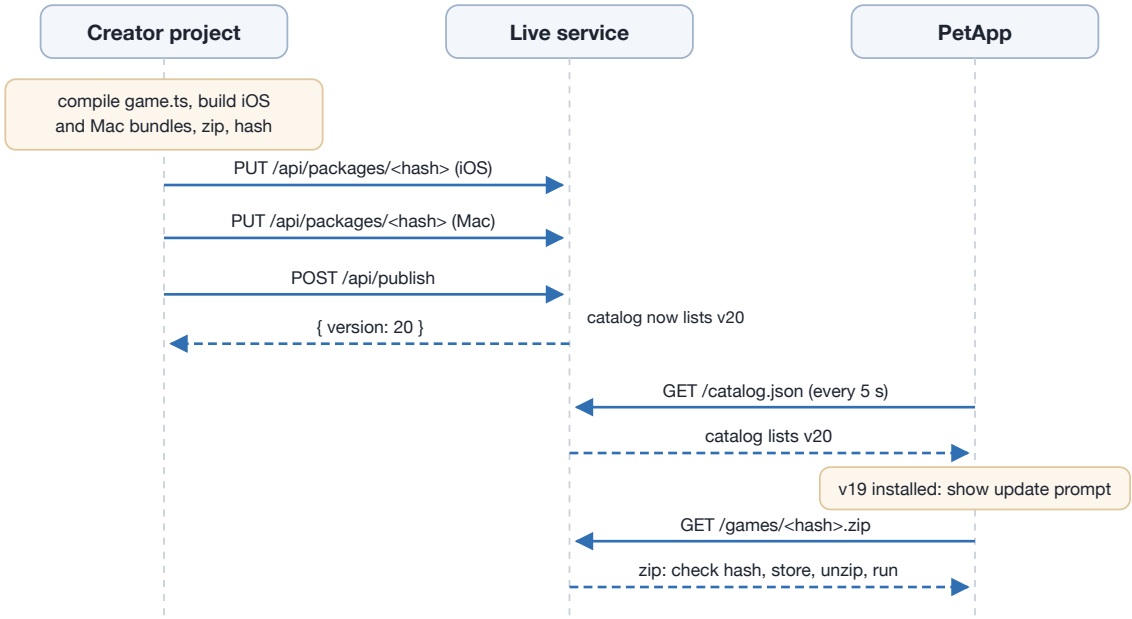


Figure 3. One publish, then one player picking up the update.

Publishing. The creator picks **Mini-game > Publish to live**. The tool compiles `game.ts` with the same `tsconfig.json` the editor uses, builds an AssetBundle for iOS and one for Mac, zips each with `game.js`, uploads both and calls publish. The new version is live as soon as the dialog says so.

Fetching. The app checks the catalog on launch, when the Games tab opens, and every 5 seconds while no game is running. When an installed game has a newer version, it asks once: *Update available, v19 → v20*. Download fetches the zip, checks the hash, unzips it and starts the game. Delete removes a game from the device; Get downloads the current version again. Offline, the app uses the last catalog it saw, and installed games stay playable.

7. What production still needs

- **Creator accounts** instead of one shared publish token, so every game has an owner.
- **Moderation before live**, plus the reporting, blocking and age-rating features from guideline 4.7.
- **Push or launch-time checks** instead of 5 second polling, which is there for the demo.
- **Rollback.** Old packages stay in R2, so going back means pointing the catalog at an older hash. There is no button for it yet.